Credit Score Classification

Pham Hoang Phuc
Faculty of Computer Science
University of Information Technology, VNU-HCM
20520278@gm.uit.edu.vn

Huynh Nguyen Van Khanh
Faculty of Computer Science
University of Information Technology, VNU-HCM
20521446@gm.uit.edu.vn

Nguyen Thi Ngoc Nga
Faculty of Computer Science
University of Information Technology, VNU-HCM
20521641@gm.uit.edu.vn

Nguyen Dang Bao Ngoc
Faculty of Computer Science
University of Information Technology, VNU-HCM
20521663@gm.uit.edu.vn

Le Tran Huu Phuoc
Faculty of Computer Science
University of Information Technology, VNU-HCM
20521775@gm.uit.edu.vn

***Abstract*—In this report, we have developed and implemented a model for categorizing individuals into credit score brackets based on their credit-related information. This task represents a classic classification problem within the field of machine learning. The model was constructed using various classification algorithms, and we performed thorough tuning and testing of hyper-parameter values for each algorithm to maximize the accuracy of the predictions.**

Subsequently, we conducted a comprehensive comparison and evaluation of these algorithms, employing different evaluation methods. The aim was to determine the most suitable algorithm for our model based on its performance. This approach allowed us to assess and select the algorithm that yielded the highest prediction results.

By implementing this model, we aim to significantly reduce manual efforts in the process of segregating individuals into credit score brackets. The model's effectiveness in accurately classifying individuals based on their credit-related information can lead to improved efficiency and reliability in credit assessments.

***Index Terms*—classification, classification algorithm, machine learning, supervised learning**

I. Introduction

A credit score is a numerical value that represents an individual's creditworthiness and their likelihood of repaying loans and fulfilling financial obligations. It is a standardized measurement used by lenders, banks, and financial institutions to assess the credit risk associated with extending credit to individuals.

Credit scores are typically calculated based on various factors, including payment history, outstanding debts, credit utilization, length of credit history, types of credit used, and recent credit inquiries. These factors provide insights into an individual's financial behavior and ability to manage credit responsibly.

For banks, credit scores are extremely useful in several ways:

- Risk Assessment: Banks use credit scores to evaluate the creditworthiness of borrowers. By examining credit scores, banks can assess the level of risk associated with granting a loan or extending credit to an individual. Higher credit scores indicate lower risk, while lower scores suggest higher risk.
- Loan Approvals: Credit scores serve as a crucial factor in determining loan approvals. Banks can set specific credit score thresholds to determine whether an individual qualifies for a loan. Those with higher credit scores are more likely to be approved, while those with lower scores may face challenges or higher interest rates.
- Interest Rates and Loan Terms: Credit scores also play a significant role in determining the interest rates and loan terms offered to borrowers. Individuals with higher credit scores are generally offered lower interest rates and more favorable loan terms, as they are considered lower risk borrowers.
- Portfolio Management: Banks can use credit scores to manage and monitor their loan portfolios effectively. By categorizing borrowers into credit score brackets, banks can analyze the risk distribution across their portfolio and make informed decisions regarding risk mitigation strategies.

Accurately classifying individuals into different credit score categories brings several benefits to banks. By automating the classification process, banks can reduce the reliance on manual assessments, leading to faster decision-making. This efficiency gains significance as it enables banks to handle larger volumes of credit evaluations and reduces the time taken to provide borrowers with loan decisions.

Our objective is to leverage the power of Machine Learning to automate the classification of individuals into credit score brackets. By harnessing the capabilities of advanced algorithms, we seek to streamline and expedite the process of categorizing individuals based on their credit-related information. This automation will enhance efficiency, reduce manual

efforts, and provide more accurate and consistent results in determining creditworthiness.

For this study, we have assigned specific individuals to experiment with each classification algorithm as follows:

- **Nguyen Dang Bao Ngoc** conducted the experiments with the K-Nearest Neighbors (KNN) algorithm.
- **Huynh Nguyen Van Khanh** conducted the experiments with the Random Forest algorithm.
- **Pham Hoang Phuc** conducted the experiments with the XGBoost algorithm.
- **Le Tran Huu Phuoc** conducted the experiments with the Light Gradient Boosting Machine (LGBM) algorithm.
- **Nguyen Thi Ngoc Nga** conducted the experiments with the Histogram Boosting Gradient Classifier.

II. Dataset

A. Overview

The dataset utilized for this study is derived from the "Credit Score classification" dataset, which was originally sourced from the website Kaggle. To ensure the dataset's suitability for our model, we conducted a thorough Exploratory Data Analysis (EDA) and made necessary adjustments to refine the data.

The dataset consists of 100,000 unique rows of credit information, encompassing details pertaining to 12,500 customers. In total, it comprises 27 features and 1 column for ground truth, which correspond to 28 columns in the dataset. These features encompass a wide range of credit-related information, offering a comprehensive view of the customers' financial profiles. The dataset comprises three distinct target labels: "Good", "Standard", and "Poor". These labels represent the credit score categories into which individuals are classified based on their credit-related information. The assignment of individuals to these categories is based on the assessment of their creditworthiness.

In addition to the training set, the dataset also includes a separate test set comprising 50,000 instances. It's important to note that this test set does not include the ground truth or target labels for each instance. Therefore, we solely utilize this test set for data cleaning purposes. We can leverage this set to identify and address missing values, outliers, or any other data inconsistencies that may impact the quality and integrity of our dataset.

By using the test set exclusively for data cleaning, we ensure that our training set remains unaffected and that the subsequent modeling and evaluation stages are conducted on a reliable and accurate dataset.

B. Exploratory Data Analysis

Understanding dataset variable

The dataset consists of 27 features, each providing valuable information for our analysis. It's important to note that some columns have missing values, including Name, Monthly Inhand Salary, Type of Loan, Number of Delayed Payment, Number of Credit Inquiries, Credit History Age, Amount Invested Monthly, and Monthly Balance.

Here is a breakdown of the columns based on their data types:

Columns with string data type:

- ID: Represents a unique identification of an entry
- Customer ID: Represents a unique identification of a person
- Month: Represents the month of the year
- Name: Represents the name of a person
- Age: Represents the age of the person
- SSN: Represents the social security number of a person
- Occupation: Represents the occupation of the person
- Annual Income: Represents the annual income of the person
- Number of Loan: Represents the number of loans taken from the bank
- Type of Loan: Represents the types of loan taken by a person
- Number of Delayed Payment: Represents the average number of payments delayed by a person
- Changed Credit Limit: Represents the percentage change in credit card limit
- Credit Mix: Represents the classification of the mix of credits
- Outstanding Debt: Represents the remaining debt to be paid (in USD)
- Credit History Age: Represents the age of credit history of the person
- Payment of Min Amount: Represents whether only the minimum amount was paid by the person
- Amount Invested Monthly: Represents the monthly amount invested by the customer (in USD)
- Payment Behavior: Represents the payment behavior of the customer (in USD)
- Monthly Balance: Represents the monthly balance amount of the customer (in USD)

Columns with numeric data type:

- Monthly In-hand Salary: Represents the monthly base salary of a person
- Number of Bank Account: Represents the number of bank accounts a person holds
- Number of Credit Card: Represents the number of other credit cards held by a person
- Interest Rate: Represents the interest rate on credit card
- Delay from Due day: Represents the average number of days delayed from the payment date
- Number of Credit Inquiries: Represents the number of credit card inquiries
- Credit Utilization Ratio: Represents the utilization ratio of credit card
- Total EMI per month: Represents the monthly EMI payments (in USD)

TABLE I
DATA SUMMARY

#	Column	Non-Null Count	Dtype
0	ID	100000	object
1	Customer_ID	100000	object
2	Month	100000	object
3	Name	90015	object
4	Age	100000	object
5	SSN	100000	object
6	Occupation	100000	object
7	Annual_Income	100000	object
8	Monthly_Inhand_Salary	84998	float64
9	Num_Bank_Accounts	100000	int64
10	Num_Credit_Card	100000	int64
11	Interest_Rate	100000	int64
12	Num_of_Loan	100000	object
13	Type_of_Loan	88592	object
14	Delay_from_due_date	100000	int64
15	Num_of_Delayed_Payment	92998	object
16	Changed_Credit_Limit	100000	object
17	Num_Credit_Inquiries	98035	float64
18	Credit_Mix	100000	object
19	Outstanding_Debt	100000	object
20	Credit_Utilization_Ratio	100000	float64
21	Credit_History_Age	90970	object
22	Payment_of_Min_Amount	100000	object
23	Total_EMI_per_month	100000	float64
24	Amount_invested_monthly	95521	object
25	Payment_Behaviour	100000	object
26	Monthly_Balance	98800	object
27	Credit_Score	100000	object

TABLE II
CONCATENATED DATA SUMMARY

#	Column	Non-Null Count	Dtype
0	ID	150000	object
1	Customer_ID	150000	object
2	Month	150000	object
3	Name	135000	object
4	Age	150000	object
5	SSN	150000	object
6	Occupation	150000	object
7	Annual_Income	150000	object
8	Monthly_Inhand_Salary	127500	float64
9	Num_Bank_Accounts	150000	int64
10	Num_Credit_Card	150000	int64
11	Interest_Rate	150000	int64
12	Num_of_Loan	150000	object
13	Type_of_Loan	132888	object
14	Delay_from_due_date	150000	int64
15	Num_of_Delayed_Payment	139500	object
16	Changed_Credit_Limit	150000	object
17	Num_Credit_Inquiries	147000	float64
18	Credit_Mix	150000	object
19	Outstanding_Debt	150000	object
20	Credit_Utilization_Ratio	150000	float64
21	Credit_History_Age	136500	object
22	Payment_of_Min_Amount	150000	object
23	Total_EMI_per_month	150000	float64
24	Amount_invested_monthly	143250	object
25	Payment_Behaviour	150000	object
26	Monthly_Balance	148238	object

Data Cleaning

To facilitate a more comprehensive data cleaning process, we will concatenate the training set and the test set. This concatenation aims to create a unified dataset for efficient data cleaning procedures. However, it is important to note that the target column (Credit Score) will not be included in this concatenated data. This omission is due to the absence of the Credit Score column in the test set. By merging the training and test sets, we can address data inconsistencies, missing values, and outliers in a more holistic manner. This unified dataset allows us to perform data cleansing operations consistently across all relevant features, improving the overall quality and reliability of the data. The concatenated dataset will solely serve as a means to enhance the data cleansing process and prepare the data for subsequent modeling and evaluation steps.

As part of the data cleansing process, several transformations are applied to ensure the data is in a suitable format for analysis. Firstly, the following columns are converted to the numeric data type: ID, Customer ID, Age, Annual Income, Number of Loan, Number of Delayed Payment, Changed Credit Limit, Outstanding Debt, Amount Invested Monthly, and Monthly Balance. In addition, the Month column values are appropriately transformed to their corresponding numeric representation. For instance, “June” is converted to 6, “July” to 7, and so on. This conversion allows for easier numerical analysis and comparison across different months. Furthermore, in the SSN column, all hyphen characters are removed from the values, and the resulting values are converted to numeric form. For the Credit History Age column, the values are transformed to represent the total number of months. For example, a value such as “15 years and 11 months” would be converted to 191 months. Lastly, the values in the Type of Loan column are converted to a string format where each type of loan is separated by a comma.

In the next step, we focus on addressing missing values (and removing outlier values for numeric data) within the dataset. Our approach involves replacing these missing values with appropriate values derived from the existing data. For non-numeric columns, missing values will be replaced with the most popular value for that specific column among instances sharing the same Customer ID. By considering instances with matching Customer IDs, we ensure that the replacement values are consistent within each customer’s data. For numeric columns, we adopt a two-step process. Initially, we determine the mode value for each Customer ID group, resulting in a set of mode values that corresponds to the unique Customer IDs in the concatenated data. From these mode values, we identify the maximum and minimum values. Subsequently, any missing or numeric value that falling outside the range defined by the maximum and minimum mode values is replaced with the mode value associated with the relevant Customer ID. This ensures that replaced values adhere to the range established by the most common values within each customer’s data.

With the exception of the “Type of Loan” column, where missing values will be replaced with the value “No Data,” the aforementioned data cleaning process will be applied to handle missing values in the dataset.

Please consult Table III for a summary of the data after

the completion of the cleaning process.

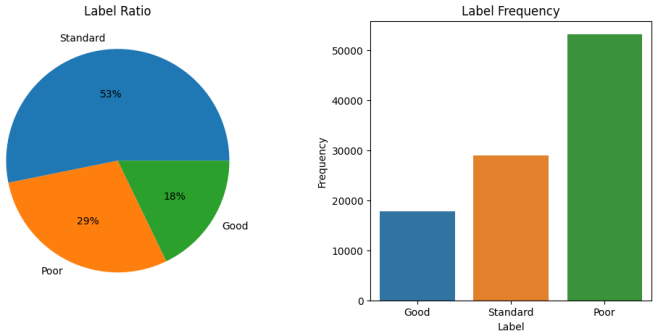
TABLE III
DATA AFTER CLEANING SUMMARY

#	Column	Non-Null Count	Dtype
0	ID	100000	int64
1	Customer_ID	100000	int64
2	Month	100000	int64
3	Name	100000	object
4	Age	100000	float64
5	SSN	100000	float64
6	Occupation	100000	object
7	Annual_Income	100000	float64
8	Monthly_Inhand_Salary	100000	float64
9	Num_Bank_Accounts	100000	float64
10	Num_Credit_Card	100000	float64
11	Interest_Rate	100000	float64
12	Num_of_Loan	100000	float64
13	Type_of_Loan	100000	object
14	Delay_from_due_date	100000	float64
15	Num_of_Delayed_Payment	100000	float64
16	Changed_Credit_Limit	100000	float64
17	Num_Credit_Inquiries	100000	float64
18	Credit_Mix	100000	object
19	Outstanding_Debt	100000	float64
20	Credit_Utilization_Ratio	100000	float64
21	Credit_History_Age	100000	float64
22	Payment_of_Min_Amount	100000	object
23	Total_EMI_per_month	100000	float64
24	Amount_invested_monthly	100000	float64
25	Payment_Behaviour	100000	object
26	Monthly_Balance	100000	float64
27	Credit_Score	100000	object

Researching dataset properties

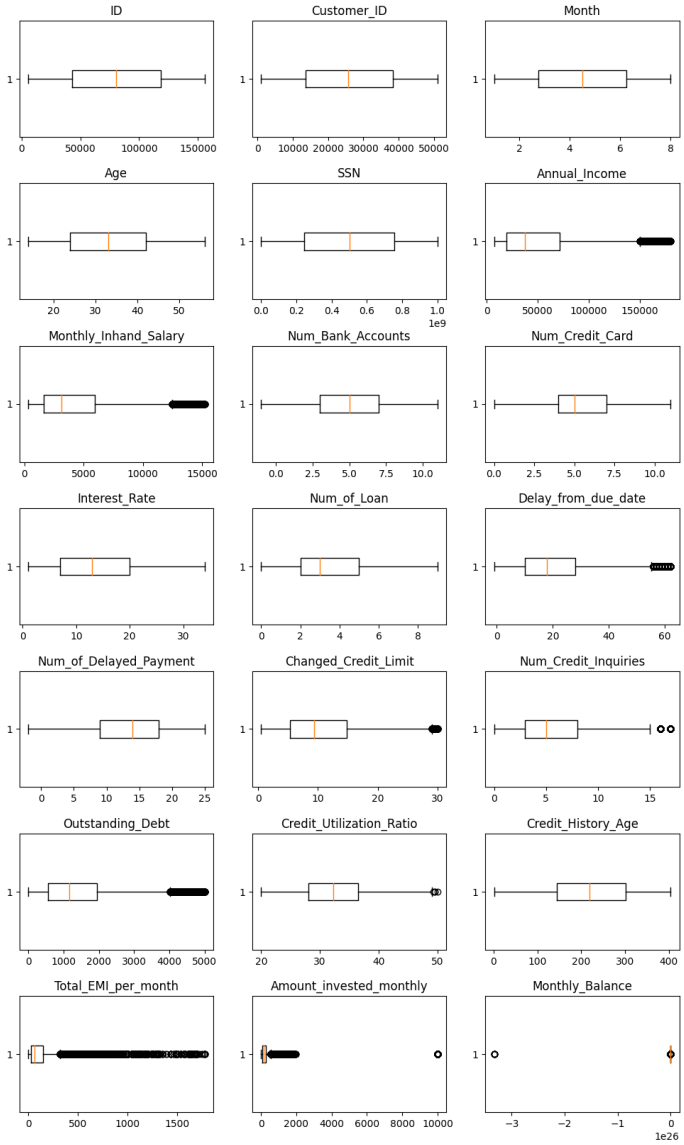
During our exploratory analysis, we uncovered two key findings regarding the dataset. Firstly, we observed an imbalance in the distribution of the target labels. Specifically, there was a noticeable disparity in the number of instances labeled as "Standard" compared to the other two categories. This imbalance highlights the need to carefully consider the impact it may have on the subsequent analysis and modeling tasks.

Fig. 1. Frequency of each label



Secondly, we identified the presence of numerous outliers in several numeric features within the dataset. However, upon closer examination, we concluded that these outliers can be attributed to income inequality. Income inequality often manifests in financial data, resulting in significant variations and extreme values in certain features. Recognizing the influence of income inequality provides valuable contextual understanding and helps explain the presence of these outliers.

Fig. 2. Boxplot of numeric features



By acknowledging these findings, we gain insights into the distribution of target labels and the nature of outliers in the dataset. These insights contribute to a more comprehensive understanding of the data and can guide us in making informed decisions during subsequent steps of the analysis.

Eliminating Unnecessary Features

Given that the Credit Score is determined based on an individual's financial status, it is unnecessary to consider their identity and personal information in this particular problem. Therefore, we will remove certain features from the dataset that solely indicate individual identity. The

features that will be eliminated include ID, Name, SSN, and Customer ID. By removing these features, we can focus exclusively on the relevant financial indicators that contribute to determining the credit score, ensuring the protection of personal information and maintaining the focus on key factors impacting creditworthiness.

Further Preprocessing

The Equated Monthly Installment (EMI) is a fixed payment amount made by a borrower to a lender on a specified date each calendar month. It includes both the interest and principal components, allowing for the gradual repayment of a loan over a specified period, including the accrued interest. When an individual's EMI significantly exceeds their monthly income, it may indicate potential financial difficulties.

To capture this information within the dataset, we introduce a new feature called the "EMI vs Income Ratio" ratio. This ratio represents the proportion of a person's monthly income that is allocated towards their EMI payment. By including this feature, we can consider the relative magnitude of an individual's EMI in relation to their income, providing insights into their financial situation. A higher EMI vs Income Ratio ratio suggests a larger portion of income being allocated towards loan repayment, potentially indicating a higher financial burden or strain.

The "Type of Loan" column contains values that are either a list of loan types separated by commas or a "No data" string. To encode this column, we represented it as a vector with a size equal to the number of unique loan types found in the dataset, plus an additional entry to account for the presence of the "No data" value. Each entry in the vector corresponds to a specific loan type.

During the encoding process, if a loan type appears in the list, the corresponding entry in the vector is encoded as 1. Conversely, if a loan type does not appear in the list, the corresponding entry in the vector is encoded as 0. This encoding approach allows us to efficiently capture the presence or absence of each loan type for every instance in a structured and interpretable format.

By transforming the "Type of Loan" column into this vector representation, we effectively represent the loan types as binary indicators, facilitating subsequent analysis and modeling tasks.

To streamline computations, we encoded the Credit Score classes as integer numbers. Specifically, we assigned the value 0 to represent the "Good" class, 1 to represent the "Poor" class, and 2 to represent the "Standard" class. This encoding allows for easier numerical processing and analysis while retaining the correspondence between the encoded values and their respective credit score categories. Additionally, we will encode all non-numeric data as integers.

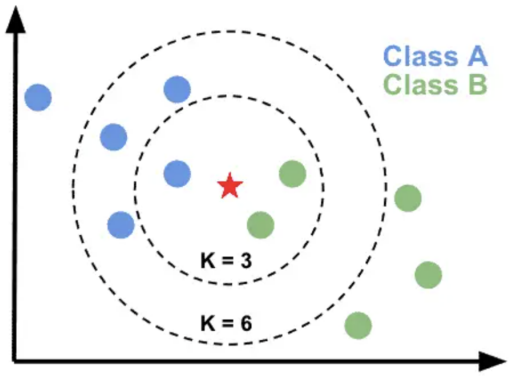
III. Methodology

In the realm of machine learning, numerous classification methods are available to tackle multi-class classification problems. In this study, we aim to conduct comprehensive experiments and analyze the final results of five classifiers: K-Nearest Neighbors, Random Forest, XGBoost, Histogram Boosting Gradient Classifier, and LGBM (Light Gradient Boosting Machine). The objective is to identify the most effective model for our specific application.

A. K-Nearest Neighbors

1) Definition: K-Nearest Neighbors (K-NN) classifies a given data point by considering the groups of its surrounding 'k' neighbors. It determines the classification based on the similarity of features between the data point and its nearest neighbors. By analyzing the feature similarity, K-NN predicts the cluster or class to which the new point is likely to belong.

Fig. 3. Example of K-Nearest Neighbors



2) Properties: K-Nearest Neighbors is a flexible algorithm used for both Classification and Regression tasks. It is characterized by its properties of being a lazy learning algorithm and a non-parametric method.

a) Lazy learning: Lazy learning refers to an algorithm that requires minimal learning time since it solely retains the training data without explicitly learning a function. The stored data is subsequently utilized to evaluate new query points. Models store the entire training dataset in memory, allowing them to make predictions or classifications by referencing and comparing the stored instances.

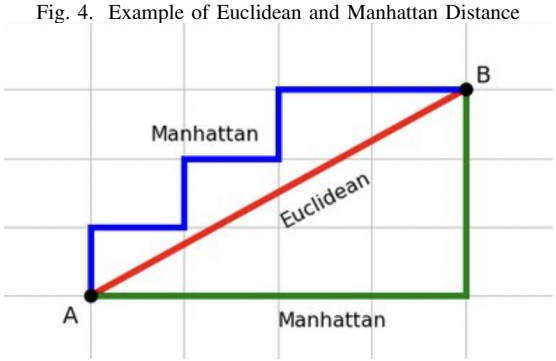
Lazy learning is distinguished by its lack of cost during the learning process, meaning it does not involve any significant computations or model building. Instead, K-NN defers the majority of the computation to the prediction phase, where distances are calculated between query instances and the stored training instances.

b) Non-parametric: The non-parametric method refers to a method that does not make any assumptions about the underlying data distribution. Unlike parametric methods, K-NN does not require estimating or finding specific parameters for the distribution. This property allows K-NN to be flexible and adaptable to various types of data without imposing any assumptions on their underlying characteristics.

3) Metric for distance computation: There are several distance metrics available for use in K-NN, including the

Euclidean Distance, Manhattan Distance, Minkowski Distance, Chebyshev Distance, and Hamming Distance. However, two of the most popular distance computation methods used in K-NN are the Euclidean distance and the Manhattan distance.

Fig. 4. Example of Euclidean and Manhattan Distance



a) Euclidean Distance: Euclidean distance calculates the straight-line distance in a multi-dimensional space. It calculates the distance between two real-valued vectors by the square root of the sum of the squared differences between the two vectors.

$$d(x, y) = \sqrt{\sum_{i=1}^{n}(x_i - y_i)^2}$$

b) Manhattan Distance: The Manhattan Distance, also known as the L1 distance or city block distance. It measures the distance between two points in a multi-dimensional space by summing the absolute differences between their corresponding coordinates.

$$d(x, y) = \sum_{i=1}^{n}|x_i - y_i|$$

4) Process: During the training phase, K-NN stores the data points from the training dataset without any explicit training or model building.

In the testing phase, the model calculates the distance between the query point and all the data points from the training phase. This distance computation is performed for each point in the test dataset, allowing the model to classify or make predictions for each test point based on the nearest neighbors from the training dataset.

After calculating the distances between the testing points and the points in the dataset, we will select the "k nearest points" in the dataset based on the calculated distances. Subsequently, we will determine the class of the testing point using a voting mechanism.

5) Pros and cons: K-Nearest Neighbors has some advantages and disadvantages

a) Advantages:

- **Simple and intuitive:** K-NN is easy to understand and implement, making it suitable for beginners in machine learning.
- **No assumptions about data distribution:** K-NN does not assume any specific data distribution, making it flexible and applicable to various types of datasets.
- **Versatile:** K-NN can be used for both classification and regression tasks, accommodating a wide range of problem domains.
- **Interpretable results:** The predictions of K-NN can often be easily interpreted and understood due to its reliance on nearest neighbors.
- **No training phase:** K-NN is a lazy learning algorithm, meaning there is no explicit training phase, resulting in faster adaptation to new data.

b) Disadvantages:

- **Computationally expensive:** K-NN can be computationally expensive, especially with large datasets, as it requires calculating distances between the query point and all training instances.
- **Sensitivity to feature scaling:** K-NN is sensitive to the scale of features, and it is crucial to normalize or scale the data beforehand to avoid biased results.
- **Need for optimal 'k' value:** The choice of the optimal value for the parameter 'k', which represents the number of neighbors, can significantly impact the performance of K-NN. Selecting an inappropriate value may lead to overfitting or underfitting.
- **Imbalanced data handling:** K-NN struggles with imbalanced datasets where the number of instances in different classes is significantly different. It may favor the majority class, resulting in biased predictions.
- **Curse of dimensionality:** K-NN can be affected by the 'curse of dimensionality' when the feature space becomes high-dimensional. In such cases, the performance of K-NN may deteriorate due to increased sparsity of data.

B. Random Forest

1) Definition: Random Forest is a machine learning algorithm that constructs a collection of decision trees during the training process. Each decision tree is built using a random subset of the training data and a random subset of input features. The predictions of the individual trees are combined to make the final prediction.

2) Ensemble Learning: Ensemble learning is a machine learning technique that involves combining the predictions of multiple individual models, called "base models," to make a final prediction or decision. The idea behind ensemble learning is that by combining the strengths of multiple models, it is possible to improve overall predictive accuracy and robustness.

a) Base models: The base models in an ensemble can be of the same type, such as multiple decision trees, or they can be different types of models altogether. The individual models are trained on different subsets of the training data or using different algorithms to introduce diversity.

b) Making predictions: The specific approach for making predictions depends on the type of ensemble learning technique employed. Here are the general steps involved:

- Voting (Classification): In ensemble learning for classification tasks, such as with Random Forest or Bagging, each base model independently predicts the class label for a given input. The ensemble then aggregates these predictions using voting. The most common types of voting are:
  - Majority Voting: The predicted class label that receives the most votes from the base models is selected as the final prediction.
  - Weighted Voting: Each base model's prediction is given a weight, and the class label with the highest weighted sum is chosen.
- Averaging (Regression): In ensemble learning for regression tasks, such as with Gradient Boosting or Stacking, each base model generates a numerical prediction for a given input. The ensemble combines these predictions through averaging methods, such as:
  - Simple Average: The predicted values from all base models are averaged to obtain the final prediction.
  - Weighted Average: Each base model's prediction is multiplied by a weight, and the weighted average of all predictions is taken.

c) Ensemble strategies: Ensemble learning techniques offer a wide range of possibilities for combining multiple models to improve predictive accuracy. While numerous approaches exist, three main classes of ensemble learning techniques are frequently discussed and widely adopted in practical applications.The three strategies are: bagging, stacking, boosting. These techniques have gained popularity primarily due to their ease of implementation and their demonstrated success across various predictive modeling problems.

3) Decision Tree: Decision tree is a supervised machine learning algorithm that uses a hierarchical structure of nodes and branches to make predictions or classify instances based on input features. It mimics the flowchart-like structure of decisions made in a tree-like model. Figure 5 illustrates the structure of a Decision Tree.

4) Properties:

a) Ensemble Learning: Random Forest is an ensemble learning method that combines multiple decision trees to make predictions. Random Forest classifier use Majority Voting mechanism for making prediction.

b) Random Subspace Sampling: During the training process, Random Forest selects a random subset of features for each tree. This technique helps to introduce diversity among the trees and reduces the chances of overfitting. It also enables the algorithm to handle high-dimensional datasets effectively.

c) Bootstrap Aggregating (Bagging): Each tree is trained on a random subset of the training data created by sampling with replacement. Figure 6 illustrates the bagging process.

d) Robust to Overfitting: Random Forest tends to be resistant to overfitting due to the randomness introduced during the training process.

5) Process:

Fig. 5. Example of a decision tree

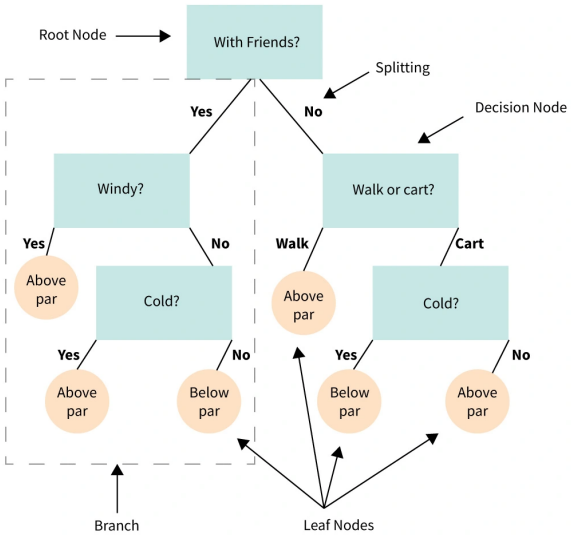
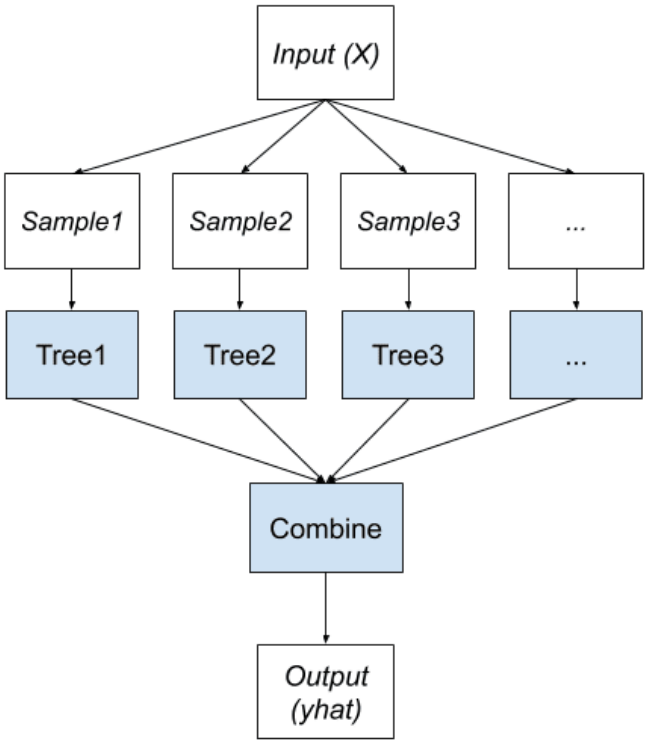


Fig. 6. Bagging Ensemble



a) Data Sampling: Through bootstrapping Random Forest create multiple datasets. Each dataset is generated by randomly sampling the original data with replacement. These bootstrapped datasets are used to train individual decision trees

b) Tree Construction: For each decision tree in the forest, a subset of features is randomly selected. The tree is then constructed only on this subset of features.

c) Splitting Nodes: At each node of a decision tree, the algorithm searches for the best feature and corresponding split point that optimizes a splitting criterion. The selected feature and split point divide the data into two child nodes.

d) Tree Growth: The splitting process continues recursively for each child node until a stopping condition is met. This condition could be reaching a maximum depth, having a minimum number of samples in a node, or other user-defined criteria.

e) Ensemble Aggregation: Once all the decision trees are grown, the Random Forest aggregates the predictions from each tree to make a final prediction.

f) Prediction: The predictions from all decision trees in the Random Forest are aggregated to obtain the final prediction. The specific aggregation method depends on the task at hand:

- Classification: In a classification problem, the most common approach is to use majority voting. The predicted class label that receives the most votes from the decision trees is selected as the final prediction.
- Regression: In a regression problem, the predicted values from all decision trees are usually averaged to obtain the final prediction.

See Figure 7 for illustration of the training process.

6) Pros and cons: Random Forest is a powerful machine learning algorithm with several advantages and disadvantages.

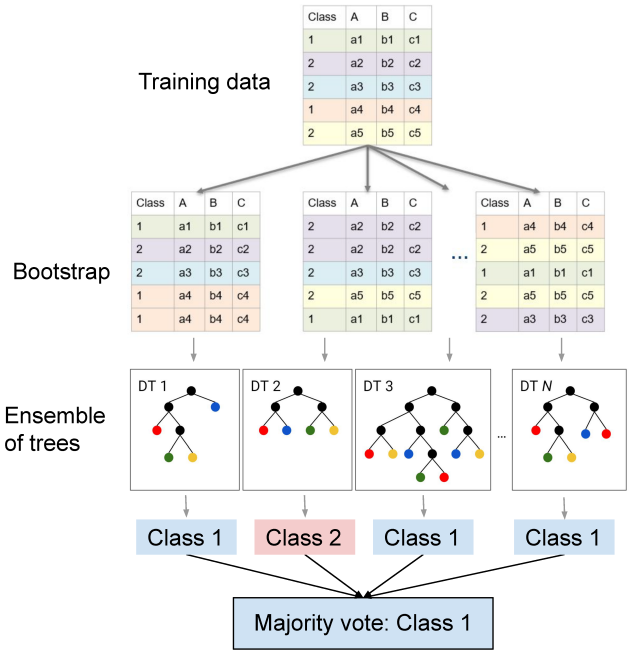
a) Advantages:

- **Robust to outliers**: Random Forest's robustness to outliers can be attributed to several factors. These include the utilization of bootstrapping and feature subsampling, the incorporation of random feature subsets during tree construction, the non-parametric nature of the algorithm, and the effectiveness of majority voting or averaging in reducing the influence of outliers within the ensemble.
- **Works well with non-linear data**: As Decision Trees are inherently non-linear algorithms, Random Forest, being an ensemble of Decision Trees, also inherits this non-linear property. The combination of multiple Decision Trees in Random Forest allows it to effectively capture and model complex non-linear relationships present in the data.
- **Lower risk of overfitting**: Random Forests are much less likely to overfit than Decision Trees because they are made up of many tree that are trained completely independently on completely different subsets of the training data.
- **Normalizing of data is not required**: Normalization of data is not necessary in Random Forest as it employs a rule-based approach.

b) Disadvantages:

- **Computational expensive**: Random Forest demands substantial computational power and resources due to its construction of multiple trees and the subsequent combination of their outputs.
- **Slow Training**: Training Random Forest also entails a significant amount of time as it combines numerous decision trees to determine the class.
- **Interpretability**: The ensemble nature of Random Forest leads to reduced interpretability and difficulty in determining the significance of each variable.

Fig. 7. Schematic structure of the Random Forest algorithm



C. XGBoost

1) Definition: XGBoost (Extreme Gradient Boosting) is a machine learning algorithm that belongs to the family of gradient boosting methods. It is designed to solve both regression and classification problems by creating an ensemble of weak prediction models, typically decision trees, and combining their predictions to obtain a more accurate and robust final model.

2) Gradient Boosting: The key concept behind XGBoost is gradient boosting, which iteratively improves the weak models by minimizing a specific loss function using gradient descent. So, what is gradient boosting?

a) Definition: Gradient boosting is a machine learning technique that combines multiple weak prediction models, typically decision trees, to create a strong predictive model. It is an ensemble learning method that iteratively builds new models to correct the errors made by previous models in the ensemble.

b) Idea: The basic idea behind gradient boosting is to optimize a specific loss function by iteratively adding new models to the ensemble. At each iteration, a new weak model is trained on the residuals (the differences between the actual target values and the predictions of the previous models). The

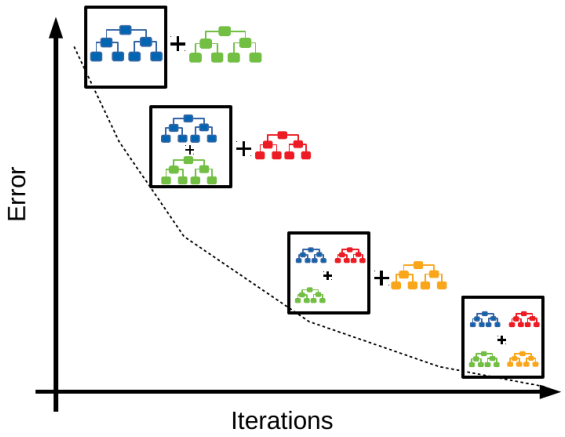
new model is then added to the ensemble, and the weights of the models are adjusted to minimize the loss function.

c) Gradient: The "gradient" in gradient boosting refers to the use of gradient descent optimization to minimize the loss function. During the training process, the algorithm calculates the gradients of the loss function with respect to the predictions of the current ensemble. These gradients provide information on how the predictions should be adjusted to reduce the loss.

d) Process: The main steps of the gradient boosting algorithm can be summarized as follows:

- **Step 1:** Initialize the ensemble by fitting an initial model (e.g., a decision tree) to the training data.
- **Step 2:** Compute the residuals by subtracting the predictions of the current ensemble from the actual target values.
- **Step 3:** Fit a new weak model to the residuals. The weak model is typically a shallow decision tree.
- **Step 4:** Adjust the weights of the models in the ensemble to minimize the loss function, using gradient descent optimization.
- **Step 5:** Repeat steps 2-4 for a specified number of iterations or until a convergence criterion is met.
- **Step 6:** The final ensemble is obtained by combining the predictions of all the models in the ensemble.

Fig. 8. Process of Gradient Boosting



3) Properties: XGBoost (Extreme Gradient Boosting) is a popular implementation of the gradient boosting algorithm that offers several distinctive properties, which contribute to its effectiveness and widespread adoption in machine learning. Some key properties of XGBoost are:

a) Scalability: XGBoost is designed to handle large-scale datasets efficiently. It supports parallel processing, enabling distributed computing and efficient utilization of multiple CPU cores. This scalability makes XGBoost suitable for handling high-dimensional datasets with millions of instances and features.

b) Regularization: XGBoost incorporates regularization techniques to prevent overfitting and improve generalization. It provides options for both L1 (Lasso) and L2 (Ridge) regularization, which control the complexity of the model by penalizing large coefficients or adding a quadratic penalty term to the loss function.

c) Tree pruning: XGBoost applies tree pruning techniques to remove unnecessary branches and nodes from decision trees, reducing their complexity and improving the model's efficiency. Pruning helps prevent overfitting and improves the model's ability to generalize to unseen data.

d) Handling missing values: XGBoost has built-in mechanisms to handle missing values within the dataset. During training, it automatically learns how to handle missing values by assigning them to the most appropriate direction in the tree construction process. This capability simplifies the data preprocessing step and allows for more robust model training.

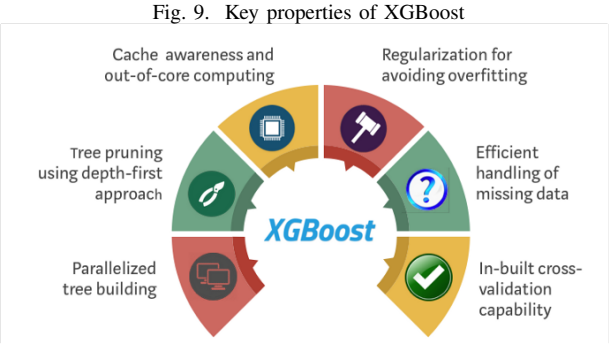
e) Feature importance: XGBoost provides a measure of feature importance, which quantifies the relative importance of each feature in contributing to the model's predictions. This information can be used for feature selection, identifying important variables, and gaining insights into the underlying relationships in the data.

f) Early stopping: XGBoost supports early stopping, which allows the training process to stop early if the model's performance on a validation set stops improving. This helps prevent overfitting and saves computation time by avoiding unnecessary iterations.

g) Handling imbalanced datasets: XGBoost includes techniques to handle imbalanced datasets, where the number of instances in different classes is significantly different. It provides options for adjusting the class weights or using different evaluation metrics to address the issue of class imbalance.

h) Interpretable outputs: XGBoost provides interpretable outputs by allowing users to examine the learned model structure, such as feature interactions and decision rules. This transparency enables better understanding and trust in the model's predictions.

Fig. 9. Key properties of XGBoost



4) Mathematical basis: Let's assume we have a training set of N samples $X = \{X_1, X_2, X_3, \ldots\}$ with corresponding target values $y = \{y_1, y_2, y_3, \ldots\}$. We will use these assumptions for the subsequent parts.

a) Process: In the first iteration, we create an initial learning tree and estimate the output values as $f_1(X)$. These estimated values will have a difference from the true values y,

which is referred to as the residual $g_1(X)$. The residual can be understood as the representation of the model's error. Our goal is to reduce this residual value to improve the performance of the model.

To achieve this, a second learning tree is built with the purpose of estimating the residuals (not the values y). Similarly, when estimating the residual $g_1(X)$, the second learning tree will estimate the value $f_2(X)$ that generates the residual $g_2(X)$. To estimate the residual $g_2(X)$, we create another learning tree. This iterative process continues.

At each iteration, a new learning tree is constructed to estimate the residuals from the previous iteration. The process continues until a predefined stopping criterion is met or a specified number of iterations is reached.

In the end, the estimated value will be the sum of the predictions from all the individual trees, denoted as $\sum f_n(X)$. This iterative process of building a sequence of weak models to correct the residuals is the fundamental principle behind gradient boosting, including XGBoost.

b) Improvements in XGBoost: To enhance the performance of the Gradient Boosting model, XGBoost introduces an additional component in the loss function. The loss function in XGBoost takes the following form:

$$J(\Theta) = L(\Theta) + \Omega(\Theta) \tag{1}$$

In which the model parameters are denoted as Θ, L represents the loss function, such as the mean squared error (MSE), Ω is the additional component often referred to as regularization, which evaluates the complexity of the model. Adding a regularization component helps balance the learned model parameters and prevents overfitting. Based on experience, using a normalized objective function like in equation (1) tends to favor simpler and more predictable models. Simpler models are better at avoiding overfitting. Since the learning model is tree-based, the final predicted values will be determined by the combination of these trees:

$$\hat{y}_i = \sum_{n=1}^{N} f_n(X_i) \tag{2}$$

Based on that, we have the specific form of the loss function at the t-th iteration as follows:

$$J_t = \sum_{i=1}^{N} L(y_i, \hat{y}_i) + \sum_{n=1}^{N} \Omega(f_n) \tag{3}$$

In XGBoost, the equation (3) can be reformulated as:

$$J_t = \sum_{i=1}^{N} \left[l(y_i, \hat{y}_i^{(t-1)}) + \sum_{k=1}^{K} w_{q(x_{ik})} \right] + \sum_{n=1}^{N} \Omega(f_n) \tag{4}$$

Here, J_t represents the loss function at the t-th iteration, $l(y_i, \hat{y}_i^{(t-1)})$ is the base loss function that measures the discrepancy between the true label y_i and the predicted value $\hat{y}_i^{(t-1)}$ at the $(t-1)$-th iteration.

The term $\sum_{k=1}^{K} w_{q(x_{ik})}$ represents the sum of the leaf weights $w_{q(x_{ik})}$ for each sample X_i in the training set. Here, $q(x_{ik})$ denotes the predicted leaf index for the i-th sample and the k-th weak model, and K represents the total number of leaf nodes in the ensemble.

The second part of the equation, $\sum_{n=1}^{N} \Omega(f_n)$, represents the regularization term for all the weak models in the ensemble. It helps control the complexity of the model and prevent overfitting.

By minimizing this loss function J_t in equation (4) with respect to the leaf weights and optimizing the regularization term, XGBoost aims to improve the overall performance of the ensemble model by iteratively updating the weak models and optimizing their leaf node assignments.

5) Pros and cons: XGBoost has several advantages and disadvantages, which are described in detail below.

a) Advantages:

- **High predictive performance:** XGBoost is widely recognized for its exceptional predictive accuracy and has achieved state-of-the-art results in various machine learning competitions and real-world scenarios.
- **Handling complex data:** XGBoost can effectively handle diverse data types, including numerical, categorical, and text data. It incorporates automatic handling of missing values and incorporates feature engineering techniques.
- **Regularization and overfitting control:** XGBoost offers built-in regularization techniques such as L1 and L2 regularization, which help prevent overfitting and improve the generalization ability of the model.
- **Feature importance estimation:** XGBoost provides a mechanism for estimating the importance of features in a dataset, enabling users to gain insights into which features significantly contribute to the predictive performance.
- **Parallel and distributed computing:** XGBoost is designed to leverage parallel and distributed computing resources, enabling faster training and prediction on large-scale datasets.

b) Disadvantages:

- **Computational complexity:** XGBoost can be computationally intensive, especially when dealing with datasets that have a large number of features and samples. Training complex models with a large number of trees may require substantial computational resources.
- **Hyperparameter tuning:** XGBoost involves several hyperparameters that need to be tuned for optimal performance. Finding the best combination of hyperparameters can be time-consuming and may require domain knowledge.
- **Interpretability:** XGBoost models are generally considered less interpretable compared to simpler models like linear regression or decision trees. The ensemble nature of XGBoost makes it challenging to interpret the impact of individual features on the predictions.
- **Data preprocessing requirements:** XGBoost performs best when the data is properly preprocessed and normalized. Handling missing values, outliers, and categorical variables may require additional preprocessing steps.

- **Model size:** XGBoost models can have a relatively large memory footprint, particularly when using a large number of trees or complex feature representations. Deploying and storing XGBoost models may require more memory resources.

D. Histogram Boosting Gradient Classifier

1) Definition: Histogram Gradient Boosting Classifier (HGBC) is a machine learning algorithm that combines the concepts of gradient boosting and histogram-based feature discretization. It is designed to handle both continuous and categorical features efficiently and has been shown to be effective in various classification tasks.

2) Gradient Boosting: To obtain detailed information on Gradient Boosting, please refer to XGBoost section of the our report just above.

3) Histogram-based feature discretization: The key concept behind Histogram Gradient Boosting Classifier (HGBC) is the use of histogram-based feature discretization and efficient computation to handle continuous features within the boosting framework. By discretizing continuous features into bins and utilizing histograms, HGBC can effectively capture the underlying patterns and relationships in the data, while also improving computational efficiency. The histograms enable efficient split point computation and provide a summarized view of feature distributions, making HGBC a powerful and scalable algorithm for classification tasks.

a) Definition: HGBC's main advantages over other boosting models include its histogram-based feature discretization, efficient handling of large-scale datasets, robustness to outliers, and interpretable models, making it stand out in terms of performance and interpretability.

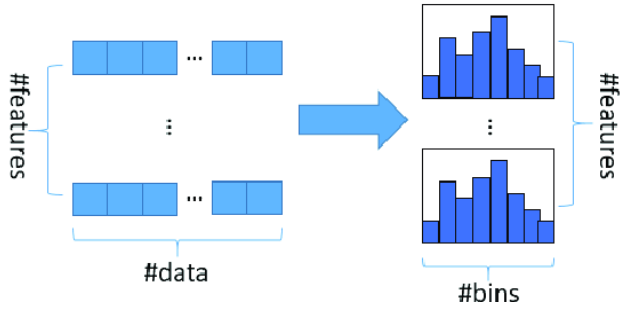
b) Idea: Histogram-based feature discretization in Histogram Gradient Boosting Classifier (HGBC) refers to the process of converting continuous features into discrete values by dividing their value ranges into bins using histograms. This technique allows HGBC to handle continuous features efficiently and incorporate them into the boosting process.

Here's an overview of how histogram-based feature discretization works in HGBC:

- **Step 1:** Binning: The first step is to divide the range of each continuous feature into a fixed number of intervals or bins. The number of bins is typically set as a hyperparameter and can be tuned based on the dataset and problem. Each bin represents a subset of the feature's values.
- **Step 2:** Histogram construction: HGBC builds histograms for each continuous feature using the bins defined in the previous step. The histograms keep track of the frequency or count of data points falling into each bin.
- **Step 3:** Discretization: For each data point, HGBC assigns it to the corresponding bin based on its feature value. This discretization process maps the continuous feature values to discrete bin labels.

By discretizing continuous features into bins, HGBC transforms them into categorical or ordinal variables. This allows HGBC to handle these features as discrete entities rather than continuous values.

Fig. 10. Histogram-based algorithm



c) Value: Histogram-based feature discretization in Histogram Gradient Boosting Classifier (HGBC) provides valuable benefits such as efficient computation, robustness to outliers, and improved interpretability. By discretizing continuous features into bins based on histograms, HGBC reduces computational complexity, handles outliers effectively, and enables the integration of mixed data types. The resulting model is more interpretable, with decision boundaries and feature importance based on discrete intervals. Overall, histogram-based feature discretization enhances the performance and interpretability of HGBC, making it a powerful algorithm for classification tasks.

4) Properties: Histogram Gradient Boosting Classifier (HGBC) exhibits several key properties that make it a powerful and desirable algorithm for classification tasks:

a) Accuracy: HGBC is known for its high predictive accuracy. By iteratively combining weak learners and adjusting weights to focus on misclassified instances, HGBC can effectively learn complex patterns in the data and make accurate predictions.

b) Efficiency: HGBC employs histogram-based feature discretization, which reduces computational complexity by summarizing the distribution of feature values within bins. This leads to faster training and prediction times, making HGBC suitable for large-scale datasets.

c) Robustness to outliers: The use of histograms in HGBC helps to handle outliers by assigning them to their respective bins. This reduces the impact of outliers on the model's decisions and enhances the overall robustness of the algorithm.

d) Interpretability: HGBC provides interpretable models. The discretization of continuous features into bins allows for easier interpretation of the model's predictions and decision boundaries. It enables the identification of important features and aids in understanding the reasoning behind the model's decisions.

e) Handling mixed data types: HGBC can effectively handle both continuous and categorical features within a unified framework. By discretizing continuous features into bins, it allows for the integration of different data types into the boosting process, providing a comprehensive solution for mixed data scenarios.

f) Handling imbalanced datasets: Boosting algorithms, including HGBC, are inherently robust to class imbalance. By adjusting weights for misclassified instances, HGBC can effectively learn from imbalanced datasets and provide accurate predictions for minority classes.

5) Pros and cons: Histogram Boosting Gradient Classifier has several advantages and disadvantages, which are described in detail below.

a) Advantages:

- **Histogram-based feature discretization:** HGBC utilizes a histogram-based approach to discretize continuous features. This technique allows for efficient handling of continuous variables by summarizing their distributions into bins. It improves computational efficiency and facilitates the integration of continuous features into the boosting process.
- **Efficient computation:** The use of histogram-based feature discretization in HGBC reduces the computational complexity compared to evaluating individual feature values. This makes HGBC particularly efficient, enabling it to handle large-scale datasets and reduce training and prediction times.
- **Robustness to outliers:** HGBC demonstrates robustness to outliers through its histogram-based feature discretization. Outliers are assigned to their respective bins, reducing their influence on the model's decisions. This robustness helps prevent overfitting and improves the generalization capability of HGBC.
- **Interpretability:** HGBC provides interpretable models, which is a valuable aspect in many domains. The discretization of features into bins enables easier interpretation of decision boundaries and feature importance. It allows users to understand and explain the reasoning behind the model's predictions.
- **Handling mixed data types:** HGBC has the ability to handle both continuous and categorical features effectively. By discretizing continuous features into bins, HGBC accommodates the integration of different data types into a unified model. This versatility makes it well-suited for datasets with mixed data scenarios.
- **Competitive performance:** HGBC has demonstrated competitive performance in various classification tasks. Its combination of efficient computation, robustness, interpretability, and the ability to handle mixed data has made it a popular choice and has achieved state-of-the-art results in different domains.

These factors collectively make HGBC special and contribute to its appeal as a powerful and versatile classifier, offering a balance between performance, efficiency, robustness, and interpretability.

b) Disadvantages: However, HGBC also has some potential disadvantages that we need to be aware of:

- **Memory usage:** HGBC can consume significant memory resources, especially when dealing with large datasets or a high number of bins in histogram-based feature discretization. The storage requirements for the histograms and ensemble of weak learners can be substantial, which may limit the scalability of the algorithm in memory-constrained environments.
- **Sensitivity to parameter tuning:** Like many machine learning algorithms, HGBC requires parameter tuning to achieve optimal performance. The performance of HGBC can be sensitive to the choice of parameters such as the number of iterations, learning rate, and depth of decision trees. Finding the best parameter configuration may require extensive experimentation and computational resources.
- **Computational complexity during training:** The training process of HGBC involves iteratively adding weak learners to the ensemble, which can be computationally expensive. Training HGBC on large datasets or with a large number of boosting iterations can take considerable time. The computational complexity can be a limitation when dealing with time-sensitive applications or large-scale datasets.
- **Lack of automatic feature interactions:** HGBC relies on individual decision trees as weak learners, which may limit its ability to capture complex feature interactions. While boosting can help mitigate this limitation to some extent, it may not capture intricate relationships between features as effectively as other models, such as deep learning algorithms.
- **Limited handling of missing data:** HGBC does not inherently handle missing data. Imputation or preprocessing techniques need to be applied beforehand to handle missing values in the dataset. The choice of imputation method can impact the performance of HGBC, and improper handling of missing data may introduce bias or inaccuracies in the model's predictions.
- **Model interpretability with high-dimensional data:** Although HGBC provides interpretable models with its histogram-based feature discretization, the interpretability may become challenging in high-dimensional datasets. As the number of features increases, understanding the influence of individual features and the decision boundaries can become more complex and difficult to interpret.

Boosting algorithms, in general, including HGBC, have been widely used and have achieved remarkable success in various machine learning competitions and challenges. Gradient Boosting algorithms, such as XGBoost and LightGBM, have been particularly popular and have won numerous competitions due to their high predictive accuracy and ability to handle complex data patterns.

While HGBC may not be as extensively studied or as widely adopted as some other boosting algorithms, its unique properties, such as histogram-based feature discretization and efficient computation, make it a promising choice for classification tasks. Its advantages in handling mixed data types, robustness to outliers, and interpretability have attracted attention from researchers and practitioners.

E. Light Gradient Boosting Machine

1) Definition: LGBM stands for LightGBM, which is a machine learning algorithm and framework developed for gradient boosting decision trees. It is designed to efficiently handle large-scale datasets and offers high training speed and memory efficiency compared to traditional gradient boosting algorithms.

It uses two novel techniques:

- Gradient-based One Side Sampling(GOSS)
- Exclusive Feature Bundling (EFB)

These techniques fulfill the limitations of the histogram-based algorithm that is primarily used in all GBDT (Gradient Boosting Decision Tree) frameworks. The two techniques of GOSS and EFB described below form the characteristics of the LightGBM Algorithm. They comprise together to make the model work efficiently and provide it a cutting edge over other GBDT frameworks.

2) Properties:

a) Gradient-based One-Side Sampling: In this section, we propose a novel sampling method for GBDT that can achieve a good balance between reducing the number of data instances and keeping the accuracy for learned decision trees.

Algorithm Description: In AdaBoost, the sample weight serves as a good indicator for the importance of data instances. However, in GBDT, there are no native sample weights, and thus the sampling methods proposed for AdaBoost cannot be directly applied. Fortunately, we notice that the gradient for each data instance in GBDT provides us with useful information for data sampling. That is, if an instance is associated with a small gradient, the training error for this instance is small and it is already well-trained. A straightforward idea is to discard those data instances with small gradients. However, the data distribution will be changed by doing so, which will hurt the accuracy of the learned model. To avoid this problem, we propose a new method called Gradient-based One-Side Sampling (GOSS). GOSS keeps all the instances with large gradients and performs random sampling on the instances with small gradients. In order to compensate the influence to the data distribution, when computing the information gain, GOSS introduces a constant multiplier for the data instances with small gradients. Specifically, GOSS firstly sorts the data instances according to the absolute value of their gradients and selects the top $a \times 100\%$ instances. Then it randomly samples $b \times 100\%$ instances from the rest of the data. After that, GOSS amplifies the sampled data with small gradients by a constant $\frac{1-a}{b}$ when calculating the information gain. By doing so, we put more focus on the under-trained instances without changing the original data distribution by much.

Theoretical Analysis:

GBDT uses decision trees to learn a function from the input space $\mathcal{X}^s$ to the gradient space $\mathcal{G}$. Suppose that we have a training set with n i.i.d. instances $\{x_1, \cdots, x_n\}$, where each x_i is a vector with dimension s in space $\mathcal{X}^s$. In each iteration of gradient boosting, the negative gradients of the loss function with respect to the output of the model are denoted as $\{g_1, \cdots, g_n\}$. The decision tree model splits each node at the most informative feature (with the largest information gain). For GBDT, the information gain is usually measured by the variance after splitting, which is defined as below.

Definition 3.1 Let O be the training dataset on a fixed node of the decision tree. The variance gain of splitting feature j at point d for this node is defined as

$$V_{j|O}(d) = \frac{1}{n_O}\left(\frac{\left(\sum_{\{x_i \in O: x_{ij} \le d\}} g_i\right)^2}{n^j_{l|O}(d)} + \frac{\left(\sum_{\{x_i \in O: x_{ij} > d\}} g_i\right)^2}{n^j_{r|O}(d)}\right)$$

where $n_O = \sum I\left[x_i \in O\right]$, $n^j_{l|O}(d) = \sum I\left[x_i \in O : x_{ij} \le d\right]$, and $n^j_{r|O}(d) = \sum I\left[x_i \in O : x_{ij} > d\right]$.

For feature j, the decision tree algorithm selects $d^*_j = \mathrm{argmax}_d V_j(d)$ and calculates the largest gain $V_j\left(d^*_j\right) \cdot$[5] Then, the data are split according feature j^* at point d_{j^*} into the left and right child nodes. In our proposed GOSS method, first, we rank the training instances according to their absolute values of their gradients in the descending order; second, we keep the top- $a \times 100\%$ instances with the larger gradients and get an instance subset A; then, for the remaining set A^c consisting $(1-a)\times 100\%$ instances with smaller gradients, we further randomly sample a subset B with size $b \times |A^c|$; finally, we split the instances according to the estimated variance gain $\tilde{V}_j(d)$ over the subset $A \cup B$, i.e.,

$$\tilde{V}_j(d) = \frac{1}{n}\left(\frac{\left(\sum_{x_i \in A_l} g_i + \frac{1-a}{b}\sum_{x_i \in B_l} g_i\right)^2}{n^j_l(d)} + \frac{\left(\sum_{x_i \in A_r} g_i + \frac{1-a}{b}\sum_{x_i \in B_r} g_i\right)^2}{n^j_r(d)}\right)$$

where $A_l = \{x_i \in A : x_{ij} \le d\}, A_r = \{x_i \in A : x_{ij} > d\}, B_l = \{x_i \in B : x_{ij} \le d\}, B_r = \{x_i \in B : x_{ij} > d\}$, and the coefficient $\frac{1-a}{b}$ is used to normalize the sum of the gradients over B back to the size of A^c.

Thus, in GOSS, we use the estimated $\tilde{V}_j(d)$ over a smaller instance subset, instead of the accurate $V_j(d)$ over all the instances to determine the split point, and the computation cost can be largely reduced. More importantly, the following theorem indicates that GOSS will not lose much training accuracy and will outperform random sampling. Due to space restrictions, we leave the proof of the theorem to the supplementary materials.

Theorem 3.2 We denote the approximation error in GOSS as $\mathcal{E}(d) = \left|\tilde{V}_j(d) - V_j(d)\right|$ and $\bar{g}^j_l(d) = \frac{\sum_{x_i \in (A \cup Ac)_l} |g_i|}{n^j_l(d)}, \bar{g}^j_r(d) = \frac{\sum_{x_i \in (A \cup A^c)_r} |g_i|}{n^j_r(d)}$. With probability at least $1-\delta$, we have

$$\mathcal{E}(d) \leq C_{a,b}^2 \ln 1/\delta \cdot \max\left\{\frac{1}{n_l^j(d)}, \frac{1}{n_r^j(d)}\right\} + 2DC_{a,b}\sqrt{\frac{\ln 1/\delta}{n}}$$

where $C_{a,b} = \frac{1-a}{\sqrt{b}} \max_{x_i \in A^c} |g_i|$, and $D = \max\left(\bar{g}_l^j(d), \bar{g}_r^j(d)\right)$.

According to the theorem, we have the following discussions: (1) The asymptotic approximation ratio of GOSS is $\mathcal{O}\left(\frac{1}{n_l^j(d)} + \frac{1}{n_r^j(d)} + \frac{1}{\sqrt{n}}\right)$. If the split is not too unbalanced (i.e., $n_l^j(d) \geq \mathcal{O}(\sqrt{n})$ and $n_r^j(d) \geq \mathcal{O}(\sqrt{n})$, the approximation error will be dominated by the second term of Ineq. (2) which decreases to 0 in $\mathcal{O}(\sqrt{n})$ with $n \to \infty$. That means when number of data is large, the approximation is quite accurate. (2) Random sampling is a special case of GOSS with $a = 0$. In many cases, GOSS could outperform random sampling, under the condition $C_{0,\beta} > C_{a,\beta-a}$, which is equivalent to $\frac{\alpha_a}{\sqrt{\beta}} > \frac{1-a}{\sqrt{\beta-a}}$ with $\alpha_a = \max_{x_i \in A \cup A^c} |g_i| \,/\, \max_{x_i \in A^c} |g_i|$.

Next, we analyze the generalization performance in GOSS. We consider the generalization error in GOSS $\mathcal{E}_{gen}^{GOSS}(d) = |V_j(d) - V_*(d)|$, which is the gap between the variance gain calculated by the sampled training instances in GOSS and the true variance gain for the underlying distribution. We have $\mathcal{E}_{\text{gen}}^{GOSS}(d) \leq \left|\tilde{V}_j(d) - V_j(d)\right| + |V_j(d) - V_*(d)| \triangleq \mathcal{E}_{GOSS}(d) + \mathcal{E}_{\text{gen}}(d)$. Thus, the generalization error with GOSS will be close to that calculated by using the full data instances if the GOSS approximation is accurate. On the other hand, sampling will increase the diversity of the base learners, which potentially help to improve the generalization performance.

b) Exclusive Feature Bundling: In this section, we propose a novel method to effectively reduce the number of features.

Algorithm 3: Greedy Bundling

Input: F: features, K: max conflict count
Construct graph G
searchOrder ← G.sortByDegree()
bundles ← {}, bundlesConflict ← {}
for i ***in*** *searchOrder* **do**
  needNew ← True
  for $j = 1$ ***to*** *len(bundles)* **do**
    cnt ← ConflictCnt(bundles[j],F[i])
    if *cnt + bundlesConflict[i]* $\leq K$ **then**
      bundles[j].add(F[i]), needNew ← False
      break
  if *needNew* **then**
    Add $F[i]$ as a new bundle to $bundles$
Output: $bundles$

Algorithm 4: Merge Exclusive Features

Input: $numData$: number of data
Input: F: One bundle of exclusive features
binRanges ← {0}, totalBin ← 0
for f ***in*** F **do**
  totalBin += f.numBin
  binRanges.append(totalBin)
newBin ← new Bin(numData)
for $i = 1$ ***to*** $numData$ **do**
  newBin[i] ← 0
  for $j = 1$ ***to*** *len(F)* **do**
    if *F[j].bin[i]* $\neq 0$ **then**
      newBin[i] ← F[j].bin[i] + binRanges[j]
Output: $newBin, binRanges$

High-dimensional data are usually very sparse. The sparsity of the feature space provides us a possibility of designing a nearly lossless approach to reduce the number of features. Specifically, in a sparse feature space, many features are mutually exclusive, i.e., they never take nonzero values simultaneously. We can safely bundle exclusive features into a single feature (which we call an exclusive feature bundle). By a carefully designed feature scanning algorithm, we can build the same feature histograms from the feature bundles as those from individual features. In this way, the complexity of histogram building changes from O (#data $\times$#feature) to O (#data $\times$#bundle), while #bundle $<<$ # feature. Then we can significantly speed up the training of GBDT without hurting the accuracy. In the following, we will show how to achieve this in detail.

There are two issues to be addressed. The first one is to determine which features should be bundled together. The second is how to construct the bundle.

3) Features:

a) Optimization in Speed and Memory Usage: Many boosting tools use pre-sort-based algorithms (e.g. default algorithm in xgboost) for decision tree learning. It is a simple solution, but not easy to optimize.

LightGBM uses histogram-based algorithms, which bucket continuous feature (attribute) values into discrete bins. This speeds up training and reduces memory usage. Advantages of histogram-based algorithms include the following:

- Reduced cost of calculating the gain for each split.
- Use histogram subtraction for further speedup.
- Reduce memory usage.
- Reduce communication cost for distributed learning.

b) Sparse Optimization: Need only O (2 $\times$# non zero data) to construct histogram for sparse features.

c) Optimization in Accuracy:

d) Optimization in Network Communication:

e) Optimization in Distributed Learning:

4) Process: The main steps of the LGBM (Light Gradient Boosting Machine) algorithm can be summarized as follows:

Step 1. Data Preparation: Preprocess the dataset by handling missing values, encoding categorical variables, and splitting the data into training and validation sets.

Step 2. Define the Objective and Evaluation Metric: Determine the objective of your LGBM model, whether it's regression or classification. Choose an appropriate evaluation metric such as mean squared error (MSE) for regression or accuracy for classification.

Step 3. Set Hyperparameters: LGBM has various hyperparameters that control its behavior during training. Set the hyperparameters such as learning rate, number of boosting iterations (number of trees), maximum tree depth, regularization parameters, and feature fraction. These hyperparameters can be tuned to optimize the model's performance.

Step 4. Initialize the Model: Create an LGBM model object and set the desired hyperparameters. You can also specify the boosting type (e.g., gradient boosting or random forests) and the objective function based on your problem type.

Step 5. Training: Train the LGBM model by fitting it to the training data. LGBM uses a gradient-based optimization technique to iteratively build decision trees. It starts with an initial prediction (e.g., mean value for regression) and sequentially adds trees to improve the predictions based on the gradients of the loss function. The algorithm follows a boosting strategy, where each subsequent tree focuses on correcting the residuals (differences between the true values and the ensemble's predictions so far).

Step 6. Cross-Validation and Early Stopping: Implement cross-validation techniques, such as k-fold cross-validation, to assess the model's performance on unseen data. Utilize early

stopping, which monitors the evaluation metric on a validation set, to stop training if the performance plateaus or deteriorates.

Step 7. Model Evaluation: Evaluate the trained LGBM model's performance using the chosen evaluation metric on the validation set. Assess how well the model generalizes to unseen data and identify potential overfitting.

Step 8. Feature Importance: Analyze the feature importance provided by LGBM to understand the relative importance of different features in the model. This information can aid in feature selection, feature engineering, and gaining insights into the dataset.

Step 9. Predictions: Use the trained LGBM model to make predictions on new, unseen data. Apply the model to the test set or real-world data to obtain predictions based on the learned patterns and relationships.

Step 10. Model Deployment: If the LGBM model meets the desired performance criteria, it can be deployed for real-world use. This involves integrating it into an application, setting up an API, or incorporating it into a larger system to make predictions on new data.

These steps provide a high-level overview of the LGBM algorithm's main components and the typical workflow. However, it's important to note that specific implementation details and considerations may vary depending on the machine learning framework or library being used.

5) Pros and cons:

a) Advantages:

- **Faster training speed and higher efficiency:** Light GBM uses a histogram-based algorithm i.e it buckets continuous feature values into discrete bins which fasten the training procedure.
- **Lower memory usage:** Replaces continuous values to discrete bins which results in lower memory usage.
- **Better accuracy than any other boosting algorithm:** It produces much more complex trees by following leaf wise split approach rather than a level-wise approach which is the main factor in achieving higher accuracy.
- **Compatibility with Large Datasets:** It is capable of performing equally well with large datasets with a significant reduction in training time as compared to XGBoost.

b) Disadvantages:

- **Overfitting:** Light GBM split the tree leaf-wise which can lead to overfitting as it produces much complex trees.
- **Compatibility with Datasets:** Light GBM is sensitive to overfitting and thus can easily overfit small data.

IV. Experiment

A. Metrics

For evaluating the performance of our models on this dataset, we employ several key metrics: accuracy, F1 score, recall, and precision. Each of these metrics provides valuable insights into different aspects of the models' classification performance.

1) Accuracy: Accuracy measures the overall correctness of the model's predictions. It calculates the percentage of correctly classified instances out of the total number of instances in the dataset. Higher accuracy indicates a better overall performance of the model in correctly predicting the credit score categories.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

2) Recall: Recall, also known as sensitivity or true positive rate, measures the model's ability to correctly identify instances belonging to a particular credit score category. It calculates the percentage of true positives out of all instances that actually belong to a specific category. Higher recall indicates that the model effectively captures instances from the respective credit score category.

$$Recall = \frac{TP}{TP + FN}$$

3) Precision: Precision measures the model's accuracy in correctly identifying instances belonging to a specific credit score category. It calculates the percentage of true positives out of all instances predicted to belong to a specific category. Higher precision indicates that the model has a lower rate of falsely labeling instances as belonging to a particular credit score category.

$$Precision = \frac{TP}{TP + FP}$$

4) F1 score: The F1 score is a combined metric that takes into account both precision and recall. It provides a balanced measure of the model's accuracy, particularly when dealing with imbalanced datasets. The F1 score considers both false positives and false negatives, making it useful for evaluating the model's performance in classifying all three credit score categories.

$$F1 = \frac{2}{precision^{-1} + recall^{-1}} = 2 \cdot \frac{precision \cdot recall}{precision + recall}$$

By considering these evaluation metrics, including accuracy, F1 score, recall, and precision, we gain a comprehensive understanding of the models' performance in correctly classifying individuals into the "Good," "Standard," and "Poor" credit score categories. These metrics collectively assess the models' predictive accuracy, ability to capture relevant instances, and the precision of their classifications.

B. Evaluation Protocol

To ensure a comprehensive and robust evaluation of our models' performance, we will utilize K-fold cross-validation with a specific value of k=10 as our evaluation protocol. K-fold cross-validation involves dividing the dataset into K equally sized folds or subsets.

In this case, with k=10, we partition the dataset into 10 folds, where 9 folds are used for training the model, and the remaining fold is reserved for validation. We iterate this process 10 times, with each fold acting as the validation set

once while the other folds serve as the training set. This ensures that every data point in the dataset is used for both training and validation, providing a thorough assessment of the model's performance.

During each iteration, we calculate the evaluation metrics including accuracy, F1 score, recall, and precision based on the predictions made on the validation fold. This allows us to gather a comprehensive evaluation of the models' performance across different subsets of the data.

By utilizing K-fold cross-validation with k=10, we can obtain a more robust estimation of the models' generalization capabilities. It helps mitigate the potential bias and variability that can arise from using a single train-test split. Furthermore, averaging the evaluation metrics over the 10 iterations provides a more reliable assessment of the models' effectiveness in accurately classifying individuals into the "Good," "Standard," and "Poor" credit score categories.

This approach enables us to assess the models' overall stability and generalization ability. By utilizing K-fold cross-validation with k=10, we ensure a rigorous evaluation of our models' performance and enhance the reliability of our findings.

It should be noted that **the evaluation protocol mentioned earlier is specifically used for comparing the final (best) models of each algorithm**. For **other comparisons conducted in this study, separate evaluation protocols will be employed**. These specific evaluation protocols will be defined within their respective sections. The rationale behind this approach is to avoid unnecessary time consumption and excessive complexity associated with applying a K-fold (k=10) evaluation for all comparisons, especially when some comparisons involve relatively straightforward tasks.

C. Hyper-parameters tuning

1) Tuning strategy: In this study, the tuning process is performed using the RandomizedSearchCV function from the sklearn library. While the parameter 'n_iter' is specifically set to 50 (except for LGBM with n_iter=30), the remaining parameters are kept at their default values.
The evaluation protocol implemented by RandomizedSearchCV incorporates **(Stratified)KFold with a value of k=5**. The performance of the cross-validated model on the test set is determined using the accuracy metric.
To compare the default and tuned models, we adopt the **train_test_split** method as our evaluation protocol, with a test split ratio of 0.2.

2) K-Nearest Neighbors:

a) Hyper-parameters: One of the most important hyper-parameters in K-NN is 'k', which represents the number of neighbors used for computation. It's important to note that if the value of 'k' is even, it might causes problems when taking a majority vote because the data has an even number of classes. To address this, it is advisable to choose an odd value of 'k' when the data has an even number of classes and an even value of 'k' when the data has an odd number of classes. This helps ensure a balanced and accurate voting process in K-NN.

To enhance the performance of the model, we utilize Random Search to find the best possible value for the hyperparameters. By exploring different configurations, Random Search helps find an optimal set of hyperparameters that maximize the performance of the K-NN model.

We specify a range of values for 'k' in Random Search from 5 to 20, with a step size of 2. This range allows us to explore a variety of values for 'k' and determine the optimal choice that yields the best results.

Moreover, Scikit-learn provides several attributes that are associated with the K-NN model. In this project, we will utilize some of these attributes to enhance our analysis and predictions:

- **n_neighbors:** int, default = 5 **(1)**
  This attribute represents the number of neighbors considered when making predictions. It specifies the value of 'k' in K-NN.
- **weights:** {'uniform', 'distance'}, callable or None, default='uniform' **(2)**
  The weights attribute determines how the contributions of neighbors are weighted during prediction. It can be set to 'uniform' where all neighbors have equal weight, or 'distance' where closer neighbors have more influence.
- **algorithm:** {'auto', 'ball_tree', 'kd_tree', 'brute'}, default = 'auto' **(3)**
  This parameter determines the algorithm used to compute nearest neighbors. The default value is 'auto', which automatically selects the most appropriate algorithm based on the data and other parameters. Other options include 'ball_tree' and 'kd_tree', which use tree-based data structures, and 'brute', which uses a brute-force approach.
- **leaf_size:** int, default=30 **(4)**
  The leaf_size parameter is specific to tree-based algorithms like 'ball_tree' and 'kd_tree'. It controls the leaf size of the tree and can affect the speed and memory usage of the algorithm.
- **p:** int, default = 2 **(5)**
  Power parameter for the Minkowski metric. When p = 1, this is equivalent to using Manhattan_distance (l1), and Euclidean_distance (l2) for p = 2. For arbitrary p, Minkowski_distance (l_p) is used.

b) Fine-tuning: However, during the process of fine-tuning, we observed that implementing normalization in K-NN can yield performance enhancements and elevate the precision of the model. We will utilize the RandomizedSearchCV tool from the sklearn library. The fine-tuning process will involve 50 iterations using the default values of the tool. The top hyperparameter combination that yields the highest score will be presented in the table IV.

As we can see from the table IV, the scores from the top 1 to top 4 are equal. However, when considering the time and accuracy for each split, there will be slight differences.

c) Comparision: In order to evaluate the effectiveness of the fine-tuning process, we will calculate and compare the

TABLE IV
TOP 5 HYPERPARAMETER COMBINATIONS WITH THE HIGHEST SCORES IN K-NN

Rank	(1)	(2)	(3)	(4)	(5)	Score
1	5	distance	brute	-	1	0.81
2	5	distance	ball_tree	30	1	0.81
3	5	distance	kd_tree	30	1	0.81
4	7	distance	kd_tree	30	1	0.81
5	9	distance	brute	-	1	0.80

accuracy, precision, recall, and F1 score values between the default parameters and the parameters obtained after fine-tuning, specifically for the highest achieved score. This evaluation will be conducted using the train-test split method, with 80,000 samples allocated for training and 20,000 samples for testing. The computed results will be presented in a table V

TABLE V
PERFORMANCE METRICS COMPARISON (HIGHEST SCORE)

Metric	Default Parameters	Tuned Parameters
Accuracy	0.76	0.818
Precision	0.73	0.807
Recall	0.75	0.818
F1 Score	0.74	0.812

The fine-tuning process has resulted in significant improvements across all metrics, resulting in significant increases in accuracy, precision, recall, and F1 scores. This improvement means that the fine-tuned parameters have successfully extracted more information from the data, allowing for more accurate predictions.

3) Random Forest:

a) Hyper-parameters: To fine-tune the model, Random Forest offers a diverse set of hyper-parameters. However, tuning all of these parameters can be a challenging and time-consuming task. As a result, we have chosen a subset of crucial hyper-parameters to focus on for the purpose of fine-tuning. These selected hyper-parameters are:

- **criterion**: The function to measure the quality of a split **(1)**
- **max_features**: The number of features to consider when looking for the best split **(2)**
  - If 'sqrt', then **max_features**=sqrt(n_features)
  - If 'log2', then **max_features**=log2(n_features)
  - If 'None', then **max_features**=n_features
- **n_estimators**: The number of trees in the forest **(3)**
- **max_depth**: The maximum depth of the tree **(4)**

b) Fine-tuning: Prior to embarking on the fine-tuning process for the chosen hyperparameters, it is imperative to acquaint ourselves with their default values. These default values for Random Forest in the credit score dataset are provided in the Table VI.

Proceeding with the fine-tuning of Random Forest, we will employ the RandomizedSearchCV tool from the sklearn library. This process will encompass 50 iterations, utilizing the default values of the tool. The objective is to identify the top 5 hyperparameter combinations that yield the highest scores. To ensure clarity and brevity, we will employ the numerical markings assigned to each hyperparameter earlier, considering their extensive full names. These results will be presented in a tabular format, denoted as Table VII.

TABLE VI
DEFAULT VALUES OF THE SELECTED HYPERPARAMETERS IN RANDOM FOREST

#	Hyperparameter	Default Value
0	criterion	gini
1	max features	sqrt
2	n_estimators	100
3	max depth	None

TABLE VII
TOP 5 HYPERPARAMETER COMBINATIONS WITH THE HIGHEST SCORES IN RANDOM FOREST

Rank	(1)	(2)	(3)	(4)	Score
1	gini	sqrt	150	50	0.823
2	gini	log2	150	40	0.822
3	gini	log2	150	50	0.821
4	entropy	log2	100	50	0.821
5	gini	sqrt	100	40	0.820

c) Comparison: In order to assess the impact of fine-tuning, we will calculate and compare accuracy, precision, recall, and F1 score metrics for both the default parameters and the optimized parameters yielding the highest score. The evaluation will be conducted using the train_test_split function, with 80,000 samples allocated for training and 20,000 samples for testing. The computed results will be presented in Table VIII, providing a comprehensive analysis of the model's performance.

TABLE VIII
PERFORMANCE METRICS COMPARISON (HIGHEST SCORE)

Metric	Default Parameters	Tuned Parameters
Accuracy	0.831	0.832
Precision	0.831	0.832
Recall	0.823	0.824
F1 Score	0.826	0.828

Upon examination, it can be observed that the results obtained from the fine-tuned model and the model using the default settings in sklearn exhibit minimal differences. This suggests that the default model performs admirably well, rendering the fine-tuning process relatively inconsequential in terms of significant improvements.

Nevertheless, from a strict perspective, the fine-tuned model does exhibit slightly superior performance compared to the default model. Considering this distinction, we will employ the fine-tuned model for the purpose of comparing different algorithms in Section D of this study. This choice ensures that the evaluation is conducted using the most optimized version of the Random Forest algorithm.

4) XGBoost:

a) Hyper-parameters: XGBoost has a wide range of hyper-parameters for model training. Tuning all of these parameters can be challenging and time-consuming. Therefore, we have selected a few key hyper-parameters for fine-tuning the model. These are:

- **reg_lambda**: Controls overfitting by penalizing weights, preventing noise fitting. **(1)**
- **reg_alpha**: Controls overfitting, promotes sparsity in feature selection. **(2)**
- **n_estimators**: Number of boosting rounds, affects model performance and training time. **(3)**
- **max_depth**: Maximum depth of each decision tree, balances complexity and overfitting. **(4)**
- **learning_rate**: Step size for boosting iterations, impacts convergence and generalization. **(5)**
- **gamma**: Minimum loss reduction for further partitioning leaf nodes, controls growth. **(6)**
- **colsample_bytree**: Randomly samples fraction of features for each tree, controls overfitting. **(7)**

By fine-tuning these important hyperparameters, we can optimize the XGBoost model's performance while keeping the tuning process manageable.

b) Fine-tuning: With the selected hyperparameters mentioned above, we will proceed with fine-tuning to find the best-performing model for the credit score dataset. However, first, let's understand the default values of these parameters. The default values of XGBoost are presented in the table IX.

TABLE IX
DEFAULT VALUES OF THE SELECTED HYPERPARAMETERS IN XGBOOST

#	Hyperparameter	Default Value
0	reg_lambda	1
1	reg_alpha	0
2	n_estimators	100
3	max_depth	6
4	learning_rate	0.3
5	gamma	0
6	colsample_bytree	1

With the selected hyperparameters and their default values mentioned above, we will proceed with fine-tuning XGBoost using random search. We will utilize the RandomizedSearchCV tool from the sklearn library. The fine-tuning process will involve 50 iterations using the default values of the tool. The top 5 hyperparameter combinations that yield the highest scores will be presented in the table X. Due to the long full names of each hyperparameter, we will use the corresponding numerical markings mentioned earlier in the hyperparameters of XGBoost.

The reason for having the same score for the top 1 to 4 entries is due to the limited precision of two decimal places used for ranking these results. However, there might be slight differences in the decimal numbers beyond the displayed digits that contribute to the ranking of these results.

c) Comparision: To evaluate the effectiveness of fine-tuning, we will compute and compare the accuracy, precision, recall, and F1 score values between the default parameters and the parameters obtained after fine-tuning for the highest score achieved. These values will be based on the train-test split method, with 80,000 samples for training and 20,000 samples for testing. The computed results will be presented in table XI.

TABLE X
TOP 5 HYPERPARAMETER COMBINATIONS WITH THE HIGHEST SCORES IN XGBOOST

Rank	(1)	(2)	(3)	(4)	(5)	(6)	(7)	Score
1	1e-05	1e-05	450	15	0.1	0.2	0.5	0.827
2	1e-05	0.1	350	18	0.1	0.4	0.7	0.826
3	1e-05	1	450	18	0.1	0.4	0.4	0.826
4	0.1	0.1	350	15	0.1	0.4	0.3	0.825
5	0.01	0.1	100	15	0.1	0.2	0.8	0.821

TABLE XI
PERFORMANCE METRICS COMPARISON (HIGHEST SCORE)

Metric	Default Parameters	Tuned Parameters
Accuracy	0.776	0.838
Precision	0.764	0.831
Recall	0.767	0.833
F1 Score	0.761	0.829

The fine-tuning process led to substantial enhancements in all metrics, resulting in a notable increase in accuracy, precision, recall, and F1 score. This improvement indicates that the tuned parameters were able to capture more information from the data and make more accurate predictions.

5) Histogram Boosting Gradient Classifier:

a) Hyper-parameters: Fine-tuning the model in HGBC involves adjusting various hyper-parameters, which can be a complex and time-consuming task. To simplify the process, we have identified a set of crucial hyper-parameters for optimization. These selected hyper-parameters include:

- **learning_rate**: The learning rate, also known as shrinkage. This is used as a multiplicative factor for the leaves values. Use 1 for no shrinkage. **(1)**
- **max_depth**: The maximum depth of each tree. **(2)**
- **l2_regularization**: The L2 regularization parameter. Use 0 for no regularization. **(3)**
- **max_leaf_nodes**: The maximum number of leaves for each tree. Must be strictly greater than 1. If None, there is no maximum limit. **(4)**
- **min_samples_leaf**: The minimum number of samples per leaf. **(5)**
- **early_stopping**: If 'auto', early stopping is enabled if the sample size is larger than 10000. If True, early stopping is enabled, otherwise early stopping is disabled. **(6)**
- **warm_start**: When set to True, reuse the solution of the previous call to fit and add more estimators to the ensemble. **(7)**

b) Fine-tuning: We will proceed with fine-tuning to discover the optimal model for the credit score dataset, using the specified hyperparameters mentioned earlier. However, before that, let's familiarize ourselves with the default values of HBGC, as depicted in the table below XII.

TABLE XII
DEFAULT VALUES OF THE SELECTED HYPERPARAMETERS IN HBGC

#	Hyperparameter	Default Value
0	learning_rate	0.1
1	max_depth	None
2	l2_regularization	0
3	max_leaf_nodes	31
4	min_samples_leaf	20
5	early_stopping	'auto'
6	warm_start	False

We will employ random search to fine-tune HBGC, utilizing the selected hyperparameters and their default values mentioned previously. To accomplish this, we will leverage the RandomizedSearchCV tool from the sklearn library. The fine-tuning procedure will encompass 50 iterations, employing the default values of the tool. The table XIII will present the top 5 hyperparameter combinations that yield the highest scores. To avoid verbosity caused by the lengthy full names of each hyperparameter, we will employ the corresponding numerical markings mentioned earlier to represent the hyperparameters of HBGC.

TABLE XIII
TOP 5 HYPERPARAMETER COMBINATIONS WITH THE HIGHEST SCORES IN HBGC

Rank	(1)	(2)	(3)	(4)	(5)	(6)	(7)	Score
1	0.2	6	0.4	45	20	False	True	0.783
2	0.5	9	0.1	20	20	False	True	0.78
3	0.5	6	0.2	25	15	True	True	0.774
4	0.5	5	0.0	30	15	True	True	0.765
5	0.5	4	0.4	20	10	True	True	0.764

c) Comparison: We will evaluate the effectiveness of the fine-tuning process by measuring and contrasting the accuracy, precision, recall, and F1 score metrics between the default parameter settings and the parameters obtained after fine-tuning, specifically focusing on the best-performing model. This assessment will be performed through a train-test split, where 80,000 samples will be used for training, and 20,000 samples will be reserved for testing. The results will be organized in XIV.

TABLE XIV
PERFORMANCE METRICS COMPARISON (HIGHEST SCORE)

Metric	Default Parameters	Tuned Parameters
Accuracy	0.76	0.78
Precision	0.755	0.777
Recall	0.747	0.769
F1 Score	0.75	0.773

The model that underwent tuning exhibits superior performance compared to the default model, as evidenced by improved metrics and a higher level of effectiveness in carrying out the intended task.

6) Light Gradient Boosting Machine:

a) Hyper-parameters: The LightGBM (LGBM) model training process involves a diverse set of hyperparameters. However, optimizing all of these parameters can present difficulties and consume a significant amount of time. Consequently, we have carefully chosen a subset of crucial hyperparameters to focus on during the fine-tuning of the model. The selected hyperparameters are as follows:

- **n_estimators**:number of boosting iterations. **(1)**
- **learning_rate**: shrinkage rate, in **dart**, it also affects on normalization weights of dropped trees. **(2)**
- **lambda_l1**: L1 regularization **(3)**
- **lambda_l2**: L2 regularization **(4)**
- **max_depth**: limit the max depth for tree model. This is used to deal with over-fitting when data is small. Tree still grows leaf-wise. **(5)**
- **subsample**: LightGBM will randomly select a subset of features on each iteration (tree) if subsample is smaller than 1.0 without resampling. **(6)**

By carefully fine-tuning these essential hyperparameters, we can achieve optimal performance for the LGBM model while ensuring that the tuning process remains manageable and efficient.

b) Fine-tuning: We will now commence the fine-tuning process using the aforementioned selected hyperparameters in order to discover the most effective model for the credit score dataset. However, before proceeding, let's familiarize ourselves with the default values of these parameters. The table below illustrates the default values of LGBM hyperparameters:

TABLE XV
DEFAULT VALUES OF THE SELECTED HYPERPARAMETERS IN LGBM

#	Hyperparameter	Default Value
0	n_estimators	100
1	learning_rate	0.1
2	lambda_l1	0.0
3	lambda_l2	0.0
4	max_depth	-1
5	subsample	1.0

We will now initiate the fine-tuning of LGBM using random search, incorporating the selected hyperparameters along with their respective default values mentioned above. To accomplish this, we will leverage the RandomizedSearchCV tool from the sklearn library. The fine-tuning process will encompass 30 iterations, utilizing the default values provided by the tool. Subsequently, the table will showcase the top 5 hyperparameter combinations that yield the highest scores. XVI. Due to the long full names of each hyperparameter, we will use the corresponding numerical markings mentioned earlier in the hyperparameters of LGBM.

TABLE XVI
TOP 5 HYPERPARAMETER COMBINATIONS WITH THE HIGHEST SCORES IN LGBM

Rank	(1)	(2)	(3)	(4)	(5)	(6)	Score
1	500	0.3	10	0	10	1	0.79
2	500	0.3	0	50	5	1	0.79
3	500	0.1	0	50	15	0.8	0.77
4	1000	0.3	20	20	10	0.6	0.77
5	1000	0.05	20	30	15	0.6	0.76

c) Comparision: To assess the efficacy of the fine-tuning process, we will calculate and compare the accuracy, precision, recall, and F1 score values between the default parameters and the parameters achieved after fine-tuning, specifically for the highest score obtained. These calculations will be conducted using the train-test split method, utilizing 80,000 samples for training and 20,000 samples for testing. The computed results will be presented in a table format for easy reference.

TABLE XVII
PERFORMANCE METRICS COMPARISON (HIGHEST SCORE)

Metric	Default Parameters	Tuned Parameters
Accuracy	0.748	0.826
Precision	0.729	0.816
Recall	0.740	0.824
F1 Score	0.733	0.820

D. Comparison

We assess the best model from each algorithm using our preprocessed data, utilizing our own evaluation protocol, and proceed to compare their performance against each other. For detailed performance information of each model, please consult XVIII.

TABLE XVIII
PERFORMANCE EVALUATION OF ALGORITHMS

#	Algorithm	Accuracy	Recall	Precision	F1
1	KNN	0.821	0.822	0.809	0.815
2	Random Forest	0.837	0.836	0.827	0.831
3	XGBoost	0.843	0.842	0.835	0.838
4	HGBC	0.828	0.822	0.827	0.817
5	LGBM	0.825	0.823	0.816	0.819

The K-Nearest Neighbors (KNN) algorithm achieved an accuracy of 0.821, with a recall of 0.822, precision of 0.809, and an F1-score of 0.815. This algorithm demonstrated competitive performance in credit score classification.

The Random Forest algorithm showed improved results with an accuracy of 0.836, recall of 0.837, precision of 0.827, and an F1-score of 0.831. It outperformed KNN and showcased its effectiveness in handling both numerical and categorical features.

XGBoost, a boosting algorithm, exhibited superior performance with an accuracy of 0.843, recall of 0.842, precision of 0.835, and an F1-score of 0.838. It demonstrated its capability to handle complex relationships in the data, resulting in accurate credit score predictions.

The Histogram-Based Gradient Boosting Classifier (HGBC) algorithm achieved an accuracy of 0.828, recall of 0.822, precision of 0.827, and an F1-score of 0.817. It showcased its ability to handle high-dimensional data efficiently and produced competitive results.

The LightGBM (LGBM) algorithm achieved an accuracy of 0.825, recall of 0.823, precision of 0.816, and an F1-score of 0.819. It demonstrated its efficiency in terms of training speed and memory consumption while providing accurate predictions.

Overall, the XGBoost algorithm exhibited the highest accuracy among the tested algorithms, followed closely by Random Forest. However, all the algorithms showcased competitive performance, indicating their potential in credit score classification tasks.

V. CONCLUSION

The experimental results demonstrated that all the tested algorithms showed promising performance in credit score classification. Each algorithm achieved a respectable level of accuracy, with XGBoost leading the pack with the highest accuracy score of 0.843. Random Forest also performed exceptionally well with an accuracy of 0.837.

Furthermore, HGBC, KNN, and LGBM also exhibited competitive performance, with accuracy scores of 0.828, 0.821, and 0.825, respectively. These algorithms demonstrated their efficacy in credit score prediction and showcased their unique strengths.

The evaluation metrics, including recall, precision, and F1-score, were also considered, indicating the algorithms' performance in correctly identifying positive and negative instances. Overall, the algorithms exhibited balanced performance across these metrics, ensuring both accurate identification of creditworthy individuals and avoiding false positives.

The findings of this study highlight the potential of machine learning algorithms in credit score classification. By leveraging these algorithms, financial institutions and credit agencies can improve the accuracy and efficiency of their credit assessment processes. Accurate credit scoring enables lenders to make informed decisions and manage risks effectively, while also ensuring fair treatment to borrowers.

ACKNOWLEDGMENT

We extend our sincere appreciation to Ph.D. Nguyen Vinh Tiep from the University of Information Technology for providing invaluable guidance and supervision to our project implementation team. We are grateful for his expertise, support, and valuable insights throughout the course of the project. His guidance has been instrumental in ensuring the successful execution of our project.

REFERENCES

[1] Credit score classification - Kaggle. (2023, June 1). Kaggle: Your Machine Learning and Data Science Community. Retrieved from https://www.kaggle.com/datasets/parisrohan/credit-score-classification

[2] What Is a Credit Score? Definition, Factors, and Ways to Raise It. (2023, June 18). Investopedia. Retrieved from https://www.investopedia.com/terms/c/credit_score.asp

[3] What is a Credit Score – Credit Score Range - Equifax. (2023, June 19). Equifax - Credit Bureau - Check Your Credit Report & Credit Score. Retrieved from https://www.equifax.com/personal/education/credit/score/what-is-a-credit-score/

[4] What is a Credit Score? - myFICO. (2023, June 18). Your FICO Score, from FICO - myFICO. Retrieved from https://www.myfico.com/credit-education/credit-scores

[5] sklearn.neighbors.KNeighborsClassifier - scikit-learn 1.3.0 documentation. (2023, June 20). scikit-learn: machine learning in Python - scikit-learn 1.3.0 documentation. Retrieved from https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

[6] What is the k-nearest neighbors algorithm? - IBM. (2023, June 21). IBM - United States. Retrieved from https://www.ibm.com/topics/knn

[7] K-Nearest Neighbor(KNN) Algorithm for Machine Learning - Javatpoint. (2023, June 21). Tutorials List - Javatpoint. Retrieved from https://www.javatpoint.com/k-nearest-neighbor-algorithm-for-machine-learning

[8] K-Nearest Neighbor(KNN) Algorithm - GeeksforGeeks. (2023, June 22). GeeksforGeeks - A computer science portal for geeks. Retrieved from https://www.geeksforgeeks.org/k-nearest-neighbours/

[9] k-NN with Hyperparameter Tuning - Kaggle. (2023, June 22). Kaggle: Your Machine Learning and Data Science Community. Retrieved from https://www.kaggle.com/code/arunimsamudra/k-nn-with-hyperparameter-tuning

[10] sklearn.ensemble.RandomForestClassifier — scikit-learn 1.3.0 documentation. (2023, June 10). scikit-learn: machine learning in Python — scikit-learn 1.3.0 documentation. Retrieved from scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html

[11] 1.11. Ensemble methods — scikit-learn 1.3.0 documentation. (2023, June 21). scikit-learn: machine learning in Python — scikit-learn 1.3.0 documentation. Retrieved from scikit-learn.org/stable/modules/ensemble.html#

[12] What is Random Forest? - IBM. (2023, June 19). IBM - United States. Retrieved from www.ibm.com/topics/random-forest#

[13] What is bagging? - IBM. (2023, June 19). IBM - United States. Retrieved from www.ibm.com/topics/bagging#

[14] Random forest - Wikipedia. (2023, June 19). Wikipedia, the free encyclopedia. Retrieved from wikipedia.org/wiki/Random_forest

[15] Bootstrap aggregating - Wikipedia. (2023, June 19). Wikipedia, the free encyclopedia. Retrieved from wikipedia.org/wiki/Bootstrap_aggregating

[16] Ensemble learning - Wikipedia. (2023, June 19). Wikipedia, the free encyclopedia. Retrieved from wikipedia.org/wiki/Ensemble_learning

[17] A Gentle Introduction to Ensemble Learning Algorithms - MachineLearningMastery.com. (2023, June 19). Machine Learning Mastery. Retrieved from machinelearningmastery.com/tour-of-ensemble-learning-algorithms/

[18] Random Forest algorithm — Machine Learning cho du lieu dang bang. (2023, June 19). Machine Learning co ban. Retrieved from machinelearningcoban.com/tabml_book/ch_model/random_forest.html

[19] Machine Learning Random Forest Algorithm - Javatpoint. (2023, June 19). Javatpoint. Retrieved from www.javatpoint.com/machine-learning-random-forest-algorithm

[20] Random Forest Classifier Tutorial - Kaggle. (2023, June 22). Kaggle: Your Machine Learning and Data Science Community. Retrieved from https://www.kaggle.com/code/prashant111/random-forest-classifier-tutorial#Random-Forest-Classifier-Tutorial-with-Python

[21] XGBoost Documentation — xgboost 1.7.6 documentation. (2023, June 16). XGBoost Documentation — xgboost 1.7.6 documentation. Retrieved from https://xgboost.readthedocs.io/en/stable/

[22] Gradient Boosting - Tat tan tat ve thuat toan manh me nhat trong Machine Learning (viblo.asia). (2023, June 17). Bai viet moi nhat - Viblo. Retrieved from https://viblo.asia/p/YWOZrN7vZQ0

[23] XGBoost: thuat toan gianh chien thang tai nhieu cuoc thi Kaggle – Ong Xuan Hong (wordpress.com). (2023, June 17). Ong Xuan Hong – Chia se kien thuc va thong tin ve Machine learning. Retrieved from https://ongxuanhong.wordpress.com/2017/12/21/xgboost-thuat-toan-gianh-chien-thang-tai-nhieu-cuoc-thi-kaggle/

[24] XGBoost Parameters Tuning - Complete Guide With Python Codes (analyticsvidhya.com). (2023, June 26). Analytics Community - Analytics Discussions - Big Data Discussion. Retrieved from https://www.analyticsvidhya.com/blog/2016/03/complete-guide-parameter-tuning-xgboost-with-codes-python/

[25] A Guide on XGBoost hyperparameters tuning — Kaggle. (2023, June 25). Kaggle: Your Machine Learning and Data Science Community. Retrieved from https://www.kaggle.com/code/prashant111/a-guide-on-xgboost-hyperparameters-tuning

[26] Histogram Boosting Gradient Classifier - Analytics Vidhya. (2023, June 28). Analytics Community - Analytics Discussions - Big Data Discussion. Retrieved from https://www.analyticsvidhya.com/blog/2022/01/histogram-boosting-gradient-classifier/

[27] Boosting Algorithms Explained. Theory, Implementation, and... - by Zixuan Zhang - Towards Data Science. (2023, June 27). Medium. Retrieved from https://towardsdatascience.com/boosting-algorithms-explained-d38f56ef3f30

[28] Histograms for efficient gradient boosting • Juliano Garcia. (2023, June 26). Juliano Garcia. Retrieved from https://robotenique.github.io/posts/gbm-histogram/

[29] The Many Flavors of Gradient Boosting Algorithms. (2023, June 28). Dataiku - Everyday AI, Extraordinary People. Retrieved from https://blog.dataiku.com/the-many-flavors-of-gradient-boosting-algorithms

[30] Welcome to LightGBM's documentation! — LightGBM 3.3.5 documentation. (2023, June 12). Welcome to LightGBM's documentation! — LightGBM 3.3.5 documentation. Retrieved from lightgbm.readthedocs.io/en/stable/

[31] Understanding LightGBM Parameters (and How to Tune Them). (2023, June 12). neptune.ai — ML metadata store. Retrieved from neptune.ai/blog/lightgbm-parameters-guide

[32] LightGBM in Python — Complete guide on how to Use LightGBM in Python. (2023, June 15).Analytics Vidhya — Learn everything about Data Science, Artificial Intelligence and Web 3.0. Retrieved from www.analyticsvidhya.com/blog/2021/08/complete-guide-on-how-to-use-lightgbm-in-python/

[33] Parameters Tuning — LightGBM 3.3.5.99 documentation. (2023, June 15).Welcome to LightGBM's documentation! — LightGBM 3.3.5 documentation. Retrieved from lightgbm.readthedocs.io/en/latest/Parameters-Tuning.html

[34] What is LightGBM, How to implement it? How to fine tune the parameters? (2023, June 15). Medium. Retrieved from medium.com/@pushkarmandot/https-medium-com-pushkarmandot-what-is-lightgbm-how-to-implement-it-how-to-fine-tune-the-parameters-60347819b7fc

[35] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, Tie-Yan Liu. (2017).LightGBM: A Highly Efficient Gradient Boosting Decision Tree. 31st Conference on Neural Information Processing Systems (NIPS 2017), Long Beach, CA, USA.

[36] 3.2. Tuning the hyper-parameters of an estimator — scikit-learn 1.3.0 documentation. (2023, June 22). scikit-learn: machine learning in Python — scikit-learn 1.3.0 documentation. Retrieved from scikit-learn.org/stable/modules/grid_search.html#randomized-parameter-search

[37] sklearn.model_selection.RandomizedSearchCV — scikit-learn 1.3.0 documentation. (2023, June 23). scikit-learn: machine learning in Python — scikit-learn 1.3.0 documentation. Retrieved from scikit-learn.org/stable/modules/generated/sklearn.model_selection.RandomizedSearchCV.html

[38] pandas documentation — pandas 2.0.3 documentation. (2023, June 21). pandas - Python Data Analysis Library. Retrieved from pandas.pydata.org/docs/

[39] NumPy user guide — NumPy v1.25 Manual. (2023, June 10). NumPy. Retrieved from numpy.org/doc/stable/user/index.html#user

[40] User guide and tutorial — seaborn 0.12.2 documentation. (2023, June 21). seaborn: statistical data visualization — seaborn 0.12.2 documentation. Retrieved from seaborn.pydata.org/tutorial.html

[41] Tutorials — Matplotlib 3.7.1 documentation. (2023, June 11). Matplotlib — Visualization with Python. Retrieved from matplotlib.org/stable/tutorials/index.html

[42] Credit Score Classification Part:1 Data Cleaning - Kaggle. (2023, June 8). Kaggle: Your Machine Learning and Data Science Community. Retrieved from www.kaggle.com/code/clkmuhammed/credit-score-classification-part-1-data-cleaning

[43] Evaluation Metrics For Classification Model — Classification Model Metrics. (2023, June 12). Analytics Vidhya — Learn everything about Data Science, Artificial Intelligence and Web 3.0. Retrieved from www.analyticsvidhya.com/blog/2021/07/metrics-to-evaluate-your-classification-model-to-take-the-right-decisions/